Exercícios de Estrutura de Dados – Lista, Fila e Pilha

Fácil

1) (Lista)Construa uma função que recebe como parâmetros uma Lista L1 e um valor x e que retire os primeiros x elementos da lista L1, inserindo-os no fim de L1. Use as funções de inserção e remoção já implementados. Suponha que a lista está inicialmente preenchida. É proibido o uso de uma lista auxiliar.

2) (Lista Arranjo) Implemente um algoritmo que retira um elemento da posição pos1 e coloca na pos2. Use os algoritmos de inserção e remoção definidos anteriormente (chame as funções implementadas no TAD Lista).

3) (Lista Arranjo) Implemente uma nova função de inserção em lista sequencial de forma que ela insira *N* cópias de um certo dado a partir de uma posição *Pos*.
void InsereNC(TipoLista *Lista, TipoItem Item, Apontador Pos, int N)

4) (Pilha Arranjo) Implemente uma nova função de desempilha, de forma que ela desenpilhe de uma só vez N elementos.
void DesempilhaNC(TipoPilha *Pilha, int N)

5) (Fila) Que conjunto de condições é necessário e suficiente para que uma sequência de operações de ***Enfileira*** **e** ***Desenfileira*** sobre uma única fila vazia deixa a fila vazia sem provocar *underflow* (tentativa de executar ***Desenfileira*** com a fila vazia)? Que conjunto de condições é necessário e suficiente para que essa sequência deixe inalterada uma fila não vazia?

6) (Pilha e Fila) Considere uma pilha P vazia e uma fila F não vazia. Utilizando apenas os testes de fila e pilha vazias, as operações **Enfileira**, **Desenfileira**, **Empilha**, **Desempilha**, e uma variável **aux** do **TipoItem**, escreva uma função que inverta a ordem dos elementos da fila.

7) (Fila) Uma fila de banco se comporta exatamente como a estrutura "fila" estudada, mas com um diferencial, o atendimento é realizado considerando uma fila preferencial e uma fila comum, as pessoas da fila preferencial são atendidas antes de qualquer outra da fila comum. Implemente uma função **atendimento**, para duas filas usando algumas das operações comuns de Enfileira, Desenfileira, e teste de vazio, sem necessidade de salvar a pessoa atendida.
void Atendimento (Tipofila *filaPref, Tipofila *filaComum)

Médio

1) (Lista Ponteiro) Construa uma função que retira um elemento de uma Lista de <u>ponteiros</u> da posição(número) X e coloca na Y. (Considere a posição 1 como primeiro elemento.)
void Troca (TipoLista* lista, int X, int Y)

2) (Fila Arranjo) Considere a implementação de filas usando arranjos "circulares". Escreva uma função **FuraFila(TipoFila* pFila, TipoItem x)** que insere um item na primeira posição da fila. O detalhe é que seu procedimento não pode movimentar os outros itens da fila. (Observe que neste caso, desrespeitaremos o conceito de FILA – primeiro a entrar é o primeiro a sair).

3) (Fila e Pilha) Como você implementaria uma fila de pilhas? Uma pilha de filas? Uma fila de filas? Escreva rotinas para implementar as operações corretas para cada uma destas estruturas de dados.

4) (Lista Ponteiro) Escreva uma função que receba como parâmetro uma lista encadeada contendo apenas valores 0 e 1 e que retorne as posições inicial e final da maior sequência S de elementos 0 dentro da lista. Ex: Lista={0,1,1,0,0,0,1,0} pini = 4 e pfim = 6 S={0,0,0} **Obs.**: No caso de empate em tamanho, a primeira ocorrência de S é a que deve ser retornada.

Difícil e/ou Grande

1) (Lista Arranjo)Escreva o código das seguintes operações adicionais para o TAD Lista Linear (supondo a implementação feita através de um vetor):
   a) Verificar se uma Lista L1 está ordenada (em ordem crescente ou decrescente)
   b) Copiar uma Lista L1 para uma outra L2. Use necessariamente a função *insere*.
   c) Copiar uma Lista L1 para uma Lista L2 eliminando elementos repetidos. Use a função *insere*.
   d) Inverter uma lista L1
   e) Intercalar L1 com L2 gerando L3 sem elementos repetidos (considere L1 e L2 ordenadas)
   f) A partir de L1 e L2, gere L4 considerando os elementos repetidos em ambas as listas (considere L1 e L2 ordenadas)

2) (Fila) Existem partes de sistemas operacionais que cuidam da ordem em que os programas devem ser executados. Por exemplo, em um sistema de computação de tempo compartilhado ("*time-shared*") existe a necessidade de manter um conjunto de processos em uma fila, esperando para serem executados. Escreva um programa que seja capaz de ler uma série de solicitações para:
    a) Incluir novos processos na fila de processo;
    b) Retirar da fila o processo com o maior tempo de espera;
    c) Imprimir o conteúdo da lista de processo em determinado momento. Assuma que cada processo é representado por um registro composto por um número identificador do processo.

3) (Fila Arranjo) Se uma fila representada por arranjos (vetores) não é considerada circular, sugere-se que cada operação ***Desenfileira*** deve deslocar para "frente" todo elemento restante de uma fila. Um método alternativo é adiar o deslocamento até que "trás" seja igual ao último índice do vetor. Quando essa situação ocorre e faz-se uma tentativa de inserir um elemento na fila, a fila inteira é deslocada para "frente", de modo que o primeiro elemento da fila fique na primeira posição do vetor, ou posição 0, caso a implementação seja em C/C++/Java. Quais são as vantagens desse método sobre um deslocamento em cada operação Desenfileira? Quais as desvantagens? Reescreva as funções ***Desenfileira***, ***Enfileira*** e ***Vazia*** usando esse novo método.

4) (Lista Ponteiro) Considere duas listas simplesmente encadeadas de números inteiros L1 e L2
    a) Crie uma lista L3 que seja a união sem repetição de L1 e L2
    b) Crie uma lista L4 que seja a intersecção dos elementos que estão em L1

5) (Fila) Construa uma função que recebe como parâmetros duas filas F1, F2 e um valor x e que retire os <u>últimos</u> x de F1, inserindo-os no <u>início</u> de F2. Use as funções de inserção e remoção já implementados e uma função int Tamanho(TipoLista lista) que retorna o tamanho de uma lista. Suponha que as listas estão inicialmente preenchidas. É proibido o uso de uma lista auxiliar ou acessar diretamente os elementos.

Referências:

1- SZWARCFITER, J.L.; MARKENZON, L. *Estruturas de Dados e Seus Algoritmos*. Editora LTC, 1998.

2- TENENBAUM, A.M.; LANGSAM Y.; AUGENSTEIN,M.A. - Estruturas de Dados em C - Prentice-Hall, 1993.

3- Lista do Prof Luis Gustavo Nonato (ICMC-USP, São Carlos), SCE5763, 2000.

4- Aaron M. Tenenbaum, Yedidyah Langsam, Moshe J. Augenstein, Estruturas de Dados Usando C, Makron Books/Pearson Education, 1995.

5- N. Ziviani, F.C. Botelho, Projeto de Algoritmos com implementações em Java e C++, Editora Thomson, 2006.